Robot Project Report

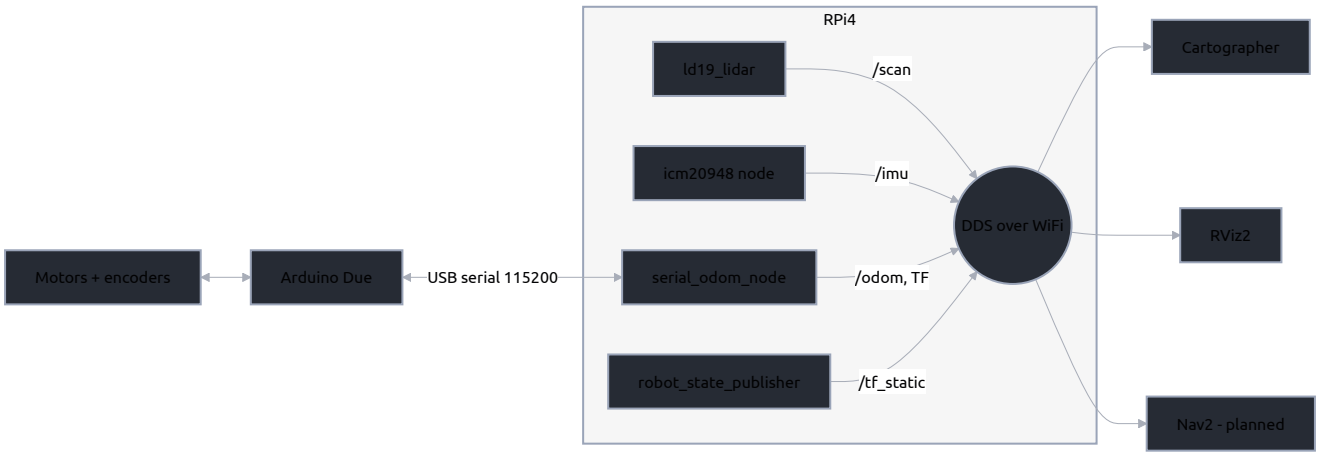
Differential-drive robot · ROS 2 Humble · RPi4 + Arduino Due + Remote PC SLAM · Status as of 30 Sep 2026

0.48 m	203	1.86 mm	10 Hz
Wheel base	Ticks / rev (single-edge)	Distance per tick	LIDAR scan rate

1. Summary

The robot is a direct-drive (no gearbox) differential platform. An RPi4 runs sensor drivers and odometry; an Arduino Due handles motor PWM and encoder counting; SLAM (Cartographer), RViz2 and planned Nav2 run on a remote PC over WiFi (`ROS_DOMAIN_ID=30` , ~2–4 ms ping). Major resolved items: encoder serial corruption in firmware, IMU I2C init failure, and RViz LIDAR drops (traced to IMU CPU load). Remaining risks are mainly **configuration inconsistencies between files** (section 6) and hardware-level items (twitch at boot, cable quality).

2. Architecture



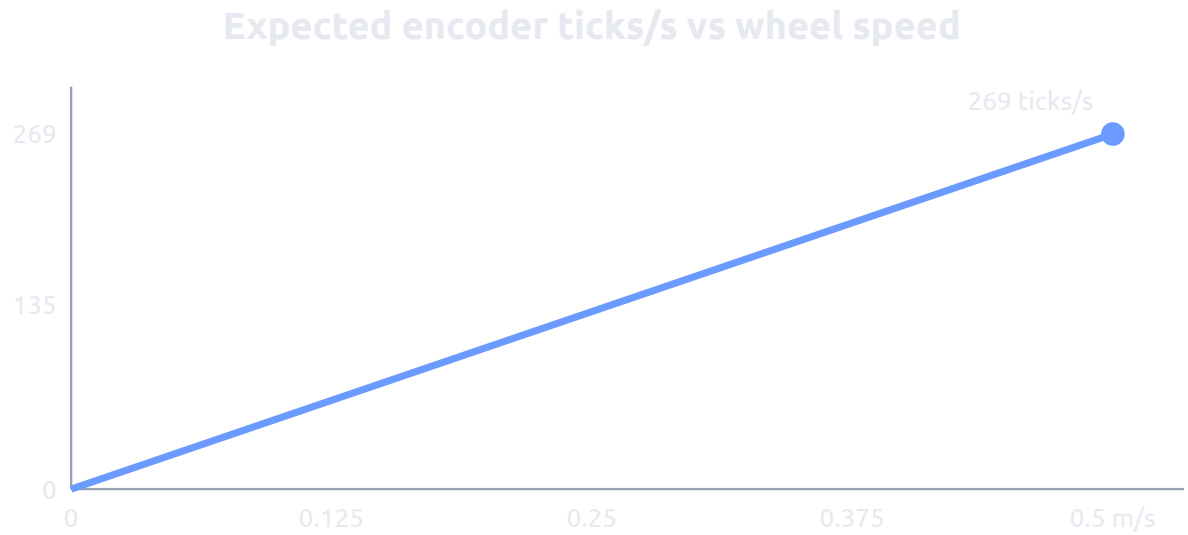
3. Hardware

Component	Detail

Compute	Raspberry Pi 4 (user `rislab`, 192.168.100.123)
Controller	Arduino Due; L298N driver (ENA/ENB PWM)
Motors	24 V DC, direct drive
Encoders	Omron E6A2-CW3C, 12 V, channel A RISING only
Wheels	Ø 12 cm, base 0.48 m
Sensors	LD19 LIDAR, ICM20948 IMU (0x69), OAK-D Lite camera
Remote PC	User `RisLab-Remote`, 192.168.100.144

4. Odometry & calibration

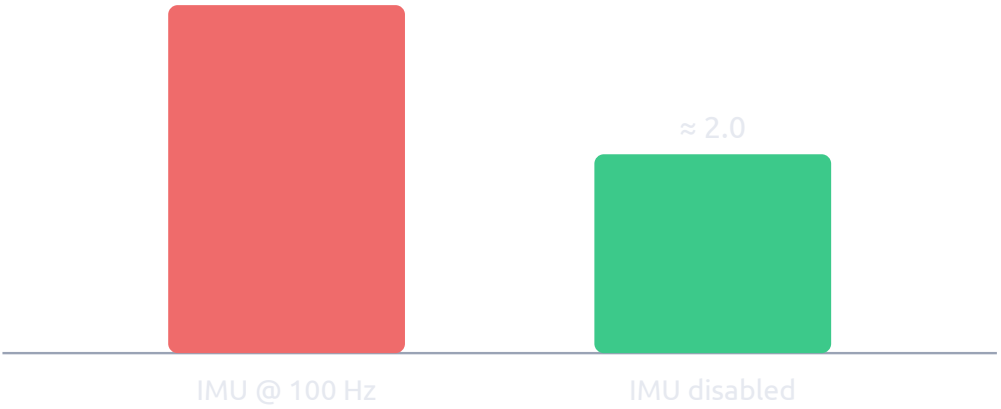
Wheel circumference = 2π·0.06 = 0.377 m, so one tick = 0.377 / 203 ≈ **1.86 mm** (≈ 538 ticks per metre). At the placeholder `MAX_WHEEL_SPEED` of 0.5 m/s, each wheel produces ≈ 269 ticks/s, or about 13 ticks per 50 ms serial packet.



Linear relation: ticks/s = v × 538. Calibration used 1 manual revolution; 5–10 revolutions divided by N would be more precise.

5. Issues found & status

RViz LIDAR drops: CPU impact of the IMU node



The IMU node used ~35% CPU (much in I/O wait) and starved DDS threads, delaying TF to the remote PC and overflowing RViz's message filter. `/scan` stayed at a clean 10 Hz, so it was never the bottleneck.

#	Issue	Status	Notes
1	Encoder serial corruption	Fixed in firmware	Print bug fixed; cable quality still suspected, test with wheels stationary
2	IMU init (Errno 121)	Resolved	Wiring / I2C fix
3	RViz queue drops	Mitigation pending	Lower IMU rate from 100 Hz (in progress)
4	Startup motor twitch	Reduced	Pins zeroed first in `setup()`; 10 kΩ pull-downs on IN1–IN4, ENA, ENB in PCB phase
5	udev symlink `/dev/turtlebot_arduino`	Unconfirmed	Node cannot open the port without it

6. Configuration review (new findings)

Cross-checking the uploaded files turned up these inconsistencies. Items are ordered by likely impact; please verify each against your actual setup.

Severity	Finding	Suggested action
High	**Duplicate odom→base_link TF.** `serial_odom_node` broadcasts it, while `cartographer.lua` sets `provide_odom_frame = true` (and `my_robot_lds_2d.lua` too).	Set `provide_odom_frame = false` when using wheel odometry, or disable the TF in the odom node.

High	**Wheel height in URDF.** Wheel joints sit at z = wheel_radius *relative to* `base_link`, which is already wheel_radius above the footprint, so axles are 0.12 m high.	Set wheel joint z to 0.
High	**Encoder voltage.** E6A2-CW3C is a 12 V device and the Due's I/O is 3.3 V (not 5-V tolerant).	Confirm a level shifter or divider and the open-collector output wiring before relying on the signal.
Med	Wheel separation mismatch: URDF 0.50 m vs node/measurement 0.48 m.	Set URDF to 0.48.
Med	Range settings differ: LD19 driver max 8.0 m; `my_robot.lua` 10 m; `my_robot_lds_2d.lua` 3.5 m. Min range 0.12 vs 0.2.	Pick one source of truth (LD19 is rated ~12 m but keep driver ≥ Cartographer).
Med	`cartographer.lua` tracks `imu_link` with `use_odometry = true` but `my_robot_lds_2d.lua` uses base_link with no IMU or odometry; two configs, two behaviours.	Decide on one config and delete the other.
Med	`ros2_control` block in the URDF names `DiffDriveController` as a hardware plugin; it is a controller, and none is launched.	Remove the block, or implement a proper hardware interface.
Low	Stale header comment in `serial_odom_node.py` says `TICKS_PER_REV` is 0; value is 203.	Update comment.
Low	Launch file uses a hard-coded path for the odom script; camera joint uses `pi` while the property is commented out (xacro's built-in `pi` works).	Package the node as an entry point.
Low	`MAX_WHEEL_SPEED = 0.5` is a guess and there is no watchdog: if `/cmd_vel` stops, the last PWM persists.	Measure max speed; add a command timeout on the Arduino or node.

7. Roadmap

Now

Lower IMU rate · confirm udev rule · resolve TF conflict · fix URDF wheel height/ separation

Next
Recalibrate `TICKS_PER_REV` over 5–10 revs · tune `MAX_WHEEL_SPEED` · add cmd_vel timeout · test serial cable · consolidate Cartographer configs · enable Nav2

Later
Move IMU to the Due (extended protocol `E,l,r,ax,ay,az,gx,gy,gz,ms`) · PCB with 10 kΩ pull-downs · remove stale duplicate files in `my_robot` (3 copies of the odom node, `nano.*.save`) and confirm `ament_cmake` vs `ament_python`

8. Key learnings

- Single-edge counting is intentional; 203 ticks/rev is not x4 quadrature.
- A healthy sensor topic rate doesn't rule out a starved DDS thread; profile CPU and D-state.